

Lattice QCD: Algorithm and Implementation Notes

Lattice_QCD

Christian Ross
Vanderbilt University

2025–2026

Contents

1	Introduction	2
2	Lattice Geometry and Gauge Fields	2
2.1	Euclidean Lattice	2
2.2	SU(3) Gauge Links	2
2.3	SU(3) Matrix Operations	3
2.4	Site Parity	3
3	Wilson Gauge Action	3
3.1	Plaquette and Action	3
3.2	Staple Sum	4
4	Gauge Field Update Algorithms	4
4.1	Heat Bath (Cabibbo–Marinari)	4
4.2	Overrelaxation	5
4.3	Metropolis	5
4.4	Combined Update	5
5	Observables: Pure Gauge	5
5.1	Plaquette and Action Density	5
5.2	Wilson Loops and Static Potential	5
5.3	Polyakov Loop	6
6	Wilson–Dirac Operator	6
6.1	Fermion Doubling and Wilson Term	6
6.2	Wilson–Dirac Operator	6
6.3	Color-Spinor Fields	6
6.4	Gamma Matrices	7

6.5	Even–Odd Preconditioning	7
7	Linear Solvers	8
7.1	Conjugate Gradient (CG)	8
7.2	BiCGstab	8
8	Quark Propagators and Meson Correlators	8
8.1	Point-to-All Propagator	8
8.2	Meson Correlators	9
8.3	Effective Mass	9
9	Hybrid Monte Carlo (HMC)	9
9.1	Overview	9
9.2	HMC Trajectory	10
9.3	Conjugate Momenta	10
9.4	Pseudofermion Generation and Action	10
10	Molecular Dynamics Forces	11
10.1	Force Convention	11
10.2	Gauge Force	11
10.3	Fermion Force	11
11	Leapfrog Integrator	12
11.1	Symplectic Integration	12
11.2	Gauge Field Update	12
11.3	Force Computation Cost	12
12	Code Architecture	13
12.1	Module Pattern	13
12.2	Source File Organization	13
12.3	OpenMP Parallelization	14
12.4	Build System	15
13	Validation	15
13.1	Pure Gauge ($\beta = 6.0$, quenched)	15
13.2	Wilson Fermions (quenched)	15
13.3	HMC ($\beta = 5.6$, $\kappa = 0.12$)	15

1 Introduction

These notes document the numerical algorithms and implementation of the `Lattice_QCD` code, a lattice quantum chromodynamics simulation in C++ with OpenMP parallelization. The code is structured following the same conventions as the `TDHFBCS_v3` code: procedural C-style C++, one function per source file, and a module pattern for global state.

The code supports the following simulation modes, in increasing order of complexity:

1. **Pure gauge (quenched) QCD:** Monte Carlo generation of SU(3) gauge field configurations via heat bath, overrelaxation, and Metropolis algorithms with the Wilson plaquette action.
2. **Quenched Wilson fermions:** Quark propagator computation on fixed gauge backgrounds using the Wilson–Dirac operator, with CG and BiCGstab solvers. Meson correlator extraction and effective mass determination.
3. **Dynamical fermions (HMC):** Hybrid Monte Carlo algorithm with pseudofermion fields for full QCD including quark loop effects.

The underlying lattice gauge theory—path integrals, Wilson’s formulation, the fermion doubling problem, and the continuum limit—is covered in the reference papers collected in `theory_notes/`. Here we focus on the numerical algorithms, discretization choices, and implementation details specific to our code.

2 Lattice Geometry and Gauge Fields

2.1 Euclidean Lattice

We work on a four-dimensional hypercubic lattice Λ with $N_x \times N_y \times N_z \times N_t$ sites and periodic boundary conditions in all directions. Lattice sites are labeled by integer coordinates $x = (x_1, x_2, x_3, x_4)$ with $x_\mu \in \{0, 1, \dots, N_\mu - 1\}$, and the lattice spacing a is set to unity throughout ($a = 1$). The lattice volume is $V = N_x N_y N_z N_t$.

Sites are mapped to a one-dimensional index via

$$\text{site}(x_1, x_2, x_3, x_4) = x_1 + N_x(x_2 + N_y(x_3 + N_z x_4)), \quad (1)$$

and neighbor tables $n_\mu^+(\text{site})$, $n_\mu^-(\text{site})$ are precomputed with periodic wrapping. These are stored in the arrays `neighborPlus[site*4+mu]` and `neighborMinus[site*4+mu]`, allocated in `lattice_module.cpp`.

2.2 SU(3) Gauge Links

The dynamical degrees of freedom are the gauge links $U_\mu(x) \in \text{SU}(3)$, stored as 3×3 complex matrices:

$$\text{gaugeField}[\text{site} * 4 + \mu] \longleftrightarrow U_\mu(x). \quad (2)$$

The total number of link variables is $4V$. Each SU(3) matrix is stored as a `struct SU3matrix` with a 3×3 array of `complex_t` elements, defined in `su3_module.hpp`.

2.3 SU(3) Matrix Operations

All SU(3) operations are free functions (no class methods). The core operations implemented in `su3_module.cpp` include:

- Multiplication, dagger (conjugate transpose), trace, determinant
- Addition, subtraction, scalar multiplication, accumulation
- Reunitarization via modified Gram–Schmidt orthonormalization
- Random SU(3) generation near identity (for Metropolis) and uniformly over the group (for hot starts)
- SU(2) subgroup heat bath (Kennedy–Pendleton algorithm) and Cabibbo–Marinari decomposition

Reunitarization is performed by Gram–Schmidt on the first two rows, followed by cross-product reconstruction of the third row to enforce $\det U = 1$.

2.4 Site Parity

The even/odd (checkerboard) decomposition is defined by the site parity

$$p(x) = (x_1 + x_2 + x_3 + x_4) \bmod 2. \quad (3)$$

Even sites ($p = 0$) and odd sites ($p = 1$) form two sublattices that do not couple directly in nearest-neighbor operations. This decomposition is used for parallel heat bath updates and even–odd preconditioning of the Dirac operator.

3 Wilson Gauge Action

3.1 Plaquette and Action

The elementary gauge-invariant object is the plaquette, a 1×1 Wilson loop:

$$P_{\mu\nu}(x) = U_\mu(x) U_\nu(x + \hat{\mu}) U_\mu^\dagger(x + \hat{\nu}) U_\nu^\dagger(x). \quad (4)$$

The Wilson gauge action is

$$S_G[U] = \frac{\beta}{N_c} \sum_x \sum_{\mu < \nu} \left[1 - \frac{1}{N_c} \text{Re Tr } P_{\mu\nu}(x) \right], \quad (5)$$

where $\beta = 2N_c/g^2$ is the inverse coupling and $N_c = 3$. The average plaquette is

$$\langle P \rangle = \frac{1}{6V} \sum_x \sum_{\mu < \nu} \frac{1}{N_c} \text{Re Tr } P_{\mu\nu}(x), \quad (6)$$

so that $S_G = 6\beta V(1 - \langle P \rangle)$. This is computed in `compute_plaquette.cpp` with OpenMP reduction over sites.

3.2 Staple Sum

The staple sum $V_\mu(x)$ collects the contributions of the six plaquettes sharing link $U_\mu(x)$:

$$V_\mu(x) = \sum_{\nu \neq \mu} \left[U_\nu(x + \hat{\mu}) U_\mu^\dagger(x + \hat{\nu}) U_\nu^\dagger(x) + U_\nu^\dagger(x + \hat{\mu} - \hat{\nu}) U_\mu^\dagger(x - \hat{\nu}) U_\nu(x - \hat{\nu}) \right]. \quad (7)$$

The first line is the “positive” staple and the second is the “negative” staple for each co-plane ν . The staple is computed in `compute_staple.cpp` and is used by all gauge update algorithms and the HMC gauge force.

4 Gauge Field Update Algorithms

4.1 Heat Bath (Cabibbo–Marinari)

The heat bath algorithm generates a new link $U_\mu(x)$ drawn from the conditional distribution

$$P(U_\mu) \propto \exp\left(\frac{\beta}{N_c} \text{Re Tr } U_\mu V_\mu^\dagger\right), \quad (8)$$

where V_μ is the staple sum. For $\text{SU}(3)$, direct sampling is not feasible, so we use the Cabibbo–Marinari method [3]: the $\text{SU}(3)$ matrix is updated by successive heat bath steps on three $\text{SU}(2)$ subgroups embedded in $\text{SU}(3)$.

For each $\text{SU}(2)$ subgroup $(i, j) \in \{(0, 1), (0, 2), (1, 2)\}$:

1. Extract the 2×2 submatrix W from $U_\mu V_\mu^\dagger$.
2. Factor $W = k w$ where $k = \sqrt{\det W}$ and $w \in \text{SU}(2)$.
3. Generate $a \in \text{SU}(2)$ from $P(a) \propto \exp(\beta_{\text{eff}} k \text{Re Tr } a)$ using the Kennedy–Pendleton algorithm [4].
4. Update: embed $a w^\dagger$ back into $\text{SU}(3)$ and left-multiply onto U_μ .

The Kennedy–Pendleton algorithm generates the $\text{SU}(2)$ element by:

1. Sample $x_0 = \cos \theta$ from $P(x_0) \propto \sqrt{1 - x_0^2} \exp(2\alpha x_0)$ where $\alpha = \beta_{\text{eff}} k$, using a rejection method with exponential envelope.
2. Sample the remaining three components uniformly on S^2 scaled by $\sqrt{1 - x_0^2}$.

Implementation: `heat_bath_sweep.cpp` performs one sweep over all links using checkerboard decomposition for OpenMP parallelism—all even-parity links are updated in parallel, then all odd-parity links.

4.2 Overrelaxation

The overrelaxation (microcanonical) update reflects the link through the local action minimum:

$$U_\mu \rightarrow V_\mu^\dagger (U_\mu^\dagger) V_\mu^\dagger \cdot (\text{normalization}), \quad (9)$$

where the SU(3) reflection is performed via Cabibbo–Marinari on SU(2) subgroups. For each subgroup, the quaternion representation is used: if $a = (a_0, \vec{a})$ and $v = (v_0, \vec{v})$, the reflected element is $a' = v^{-2}a$ in quaternion arithmetic, where v is the SU(2) projection of $UV^\dagger$.

Overrelaxation preserves the action exactly (microcanonical) but accelerates decorrelation by a factor of ~ 2 –3. A typical update cycle combines 1 heat bath sweep with $n_{\text{OR}} = 3$ –5 overrelaxation sweeps. Implementation: `overrelaxation_sweep.cpp`.

4.3 Metropolis

The Metropolis algorithm proposes a random SU(3) matrix near identity:

$$U'_\mu = R \cdot U_\mu, \quad R = \exp(i\epsilon H), \quad (10)$$

where H is a random traceless Hermitian matrix with elements of order ϵ . The proposal is accepted with probability $\min(1, e^{-\Delta S})$ where $\Delta S = S[U'] - S[U]$. The parameter ϵ is tuned to achieve $\sim 50\%$ acceptance. Implementation: `metropolis_sweep.cpp`.

4.4 Combined Update

The function `gauge_update_combined.cpp` performs one combined update cycle: 1 heat bath sweep followed by `nOverrelax` overrelaxation sweeps. This is called during both thermalization (`thermalization.cpp`) and production (`generate_configurations.cpp`).

5 Observables: Pure Gauge

5.1 Plaquette and Action Density

The average plaquette $\langle P \rangle$ (Eq. 6) is the primary diagnostic. At $\beta = 6.0$ for SU(3), the equilibrium value is $\langle P \rangle \approx 0.59$ –0.60. The action density is $s = 6\beta(1 - \langle P \rangle)$.

5.2 Wilson Loops and Static Potential

The $R \times T$ Wilson loop is a rectangular path-ordered product:

$$W(R, T) = \frac{1}{N_c} \left\langle \text{Re Tr} \prod_{\text{path}} U_\mu \right\rangle, \quad (11)$$

averaged over all spatial positions and orientations. For large T :

$$W(R, T) \sim \exp(-V(R)T), \quad (12)$$

so the static quark–antiquark potential is extracted as

$$V(R) = -\ln \frac{W(R, T)}{W(R, T-1)}. \quad (13)$$

The Cornell potential $V(R) = -A/R + \sigma R + C$ exhibits Coulomb behavior at short range and linear confinement at long range. Implementation: `wilson_loop.cpp`, `static_potential.cpp`.

5.3 Polyakov Loop

The Polyakov loop is the trace of the path-ordered product of temporal links wrapping around the lattice:

$$L(\vec{x}) = \frac{1}{N_c} \text{Tr} \prod_{t=0}^{N_t-1} U_4(\vec{x}, t). \quad (14)$$

The spatial average $\langle L \rangle$ is an order parameter for the deconfinement phase transition: $\langle L \rangle = 0$ in the confined phase and $\langle L \rangle \neq 0$ in the deconfined phase. Implementation: `polyakov_loop.cpp`.

6 Wilson–Dirac Operator

6.1 Fermion Doubling and Wilson Term

Naive lattice fermions produce $2^4 = 16$ degenerate fermion species (doublers) due to the periodicity of the lattice Brillouin zone. Wilson’s solution [2] adds a dimension-5 operator—the Wilson term—that gives the doublers a mass of order $1/a$, decoupling them in the continuum limit at the cost of explicitly breaking chiral symmetry.

6.2 Wilson–Dirac Operator

The Wilson–Dirac operator acting on a color-spinor field $\psi(x)$ is

$$\begin{aligned} (D_W \psi)(x) = \psi(x) - \kappa \sum_{\mu=1}^4 & \left[(r - \gamma_\mu) U_\mu(x) \psi(x + \hat{\mu}) \right. \\ & \left. + (r + \gamma_\mu) U_\mu^\dagger(x - \hat{\mu}) \psi(x - \hat{\mu}) \right], \end{aligned} \quad (15)$$

where $\kappa = 1/(2m_0 + 8r)$ is the hopping parameter, $r = 1$ is the Wilson parameter, m_0 is the bare quark mass, and γ_μ are the Euclidean Dirac matrices. The critical hopping parameter κ_c is the value at which the pion mass vanishes.

6.3 Color-Spinor Fields

A color-spinor field $\psi(x)$ has 4 Dirac (spin) and 3 color components at each lattice site:

$$\psi(x)_{s,c}, \quad s \in \{0, 1, 2, 3\}, \quad c \in \{0, 1, 2\}. \quad (16)$$

This is stored as `struct ColorSpinor { complex_t v[4][3]; }` defined in `colorspinor_module.hpp`. Field-level BLAS-like operations (dot product, norm, axpy, copy) are implemented with OpenMP in `colorspinor_module.cpp`.

6.4 Gamma Matrices

We use the DeGrand–Rossi (chiral) basis for the Euclidean gamma matrices. In this basis, each γ_μ has exactly one nonzero entry per row, enabling an efficient sparse representation:

$$(\gamma_\mu)_{s,s'} = g_\mu^{\text{val}}(s) \delta_{s',g_\mu^{\text{idx}}(s)}, \quad (17)$$

stored as arrays `gammaIdx[5][4]` and `gammaVal[5][4]` (indices 0–3 for γ_1 – γ_4 , index 4 for γ_5). The γ_5 matrix is diagonal: $\gamma_5 = \text{diag}(+1, +1, -1, -1)$.

The Wilson–Dirac operator application involves, for each site and direction μ : color-rotating the neighbor field by U_μ or $U_\mu^\dagger$, then applying $(r \mp \gamma_\mu)$ using the sparse representation. Implementation: `wilson_dirac_operator.cpp`. The adjoint operator $D_W^\dagger = \gamma_5 D_W \gamma_5$ is implemented in `wilson_dirac_dagger.cpp`.

6.5 Even–Odd Preconditioning

The Wilson–Dirac operator can be written in block form on the even/odd sublattices:

$$D_W = \begin{pmatrix} 1 & -\kappa D_{eo} \\ -\kappa D_{oe} & 1 \end{pmatrix}, \quad (18)$$

where D_{eo} and D_{oe} are the hopping terms between sublattices. The Schur complement

$$\hat{D} = 1 - \kappa^2 D_{eo} D_{oe} \quad (19)$$

acts only on even sites, halving the system size. After solving $\hat{D}\psi_e = b_e$, the odd-site solution is reconstructed as $\psi_o = b_o + \kappa D_{oe}\psi_e$. Implementation: `even_odd_preconditioning.cpp`.

7 Linear Solvers

7.1 Conjugate Gradient (CG)

The CG algorithm solves the normal equations $D_W^\dagger D_W x = D_W^\dagger b$, exploiting the fact that $D_W^\dagger D_W$ is Hermitian positive-definite. Given initial guess $x_0 = 0$:

$$\begin{aligned}
 r_0 &= D_W^\dagger b, \quad p_0 = r_0, \quad \rho_0 = \|r_0\|^2 \\
 \text{For } k &= 0, 1, 2, \dots : \\
 q_k &= D_W^\dagger D_W p_k \\
 \alpha_k &= \rho_k / (p_k, q_k) \\
 x_{k+1} &= x_k + \alpha_k p_k \\
 r_{k+1} &= r_k - \alpha_k q_k \\
 \rho_{k+1} &= \|r_{k+1}\|^2 \\
 \beta_k &= \rho_{k+1} / \rho_k \\
 p_{k+1} &= r_{k+1} + \beta_k p_k
 \end{aligned} \tag{20}$$

Convergence criterion: $\|r_k\|/\|r_0\| < \epsilon_{\text{CG}}$ (default 10^{-8}). Each iteration requires one application of D_W and one of $D_W^\dagger$. Implementation: `cg_solver.cpp`.

7.2 BiCGstab

The BiCGstab (biconjugate gradient stabilized) algorithm [6] solves the non-symmetric system $D_W x = b$ directly, which can be more efficient than CG on the normal equations since it requires only one D_W application per iteration (rather than $D_W + D_W^\dagger$).

A key implementation detail: the shadow residual $\hat{r}_0$ must be chosen carefully. Using $\hat{r}_0 = r_0 = b$ (the standard choice) causes breakdown for sparse sources (point sources) because $(\hat{r}_0, v) = (b, D_W p)$ can vanish when b has support on a single site. Our implementation uses a **random shadow residual** generated from a fixed seed, which eliminates this breakdown. Implementation: `bicgstab_solver.cpp`.

8 Quark Propagators and Meson Correlators

8.1 Point-to-All Propagator

The quark propagator $S(x; x_0)$ from source point x_0 is obtained by solving

$$D_W S_{s_0, c_0}(x) = \eta_{s_0, c_0}(x), \quad \eta_{s_0, c_0}(x) = \delta_{x, x_0} \delta_{s, s_0} \delta_{c, c_0}, \tag{21}$$

for all 12 source spin-color combinations (s_0, c_0) . This requires 12 linear solves per gauge configuration. Implementation: `compute_propagator.cpp`, `point_source.cpp`.

8.2 Meson Correlators

The zero-momentum meson correlator for channel Γ is

$$C_\Gamma(t) = - \sum_{\vec{x}} \text{Tr} [\Gamma S(\vec{x}, t; 0) \Gamma^\dagger \gamma_5 S^\dagger(\vec{x}, t; 0) \gamma_5], \quad (22)$$

where the trace is over spin and color, and γ_5 -hermiticity ($S^\dagger = \gamma_5 S \gamma_5$) is used to avoid computing the backward propagator. The channels implemented are:

Channel	Γ	J^{PC}
Pion	γ_5	0^{-+}
Rho (x, y, z)	γ_i	1^{--}
Scalar	$\mathbf{1}$	0^{++}
$a_1(x, y, z)$	$\gamma_5 \gamma_i$	1^{++}

A subtlety in the implementation: $\Gamma^\dagger$ acts on the *source* spin index of the propagator, mixing different source-spin components. For the pion ($\gamma_5^\dagger \gamma_5 = \mathbf{1}$), the source spin is unchanged, but for the rho and a_1 channels, the index mapping $s_0 \rightarrow a = (\gamma_\Gamma^{\text{idx}})(s_0)$ must be applied. Implementation: `meson_correlator.cpp`.

8.3 Effective Mass

The effective mass is extracted from the correlator ratio:

$$m_{\text{eff}}(t) = \ln \frac{C(t)}{C(t+1)}, \quad (23)$$

which approaches a plateau $m_{\text{eff}} \rightarrow m$ at large t where the ground state dominates. A cosh variant accounting for backward-propagating states on a periodic lattice is also available:

$$\frac{C(t)}{C(t+1)} = \frac{\cosh(m(t - N_t/2))}{\cosh(m(t+1 - N_t/2))}, \quad (24)$$

solved by Newton's method. Implementation: `effective_mass.cpp`.

9 Hybrid Monte Carlo (HMC)

9.1 Overview

The HMC algorithm [5] generates gauge configurations distributed according to the full QCD partition function including dynamical fermion effects:

$$Z = \int \mathcal{D}U \det(D_W^\dagger D_W) e^{-S_G[U]}. \quad (25)$$

The fermion determinant is represented using pseudofermion fields ϕ :

$$\det(D_W^\dagger D_W) = \int \mathcal{D}\phi^\dagger \mathcal{D}\phi \exp\left(-\phi^\dagger (D_W^\dagger D_W)^{-1} \phi\right). \quad (26)$$

HMC introduces conjugate momenta $H_\mu(x)$ (traceless anti-Hermitian matrices in the Lie algebra $\mathfrak{su}(3)$) and evolves the system via molecular dynamics (MD) followed by a Metropolis accept/reject step.

9.2 HMC Trajectory

A single HMC trajectory consists of the following steps, implemented in `hmc_trajectory.cpp`:

1. **Save** the current gauge field $U_\mu^{(0)}$.
2. **Generate momenta**: draw $H_\mu(x)$ from $P(H) \propto \exp(-\frac{1}{2} \sum |H_{ij}|^2)$, i.e., Gaussian-distributed traceless anti-Hermitian matrices.
3. **Generate pseudofermion**: draw Gaussian ξ and set $\phi = D_W^\dagger \xi$. Then $S_{\text{PF}} = \phi^\dagger (D_W^\dagger D_W)^{-1} \phi = \|\xi\|^2$ initially.
4. **Compute initial Hamiltonian**: $\mathcal{H}_i = T + S_G + S_{\text{PF}}$ where $T = \frac{1}{2} \sum_{\mu,x} |H_\mu(x)|^2$.
5. **Leapfrog integration**: evolve (U, H) for N_{MD} steps (Section 11).
6. **Compute final Hamiltonian**: $\mathcal{H}_f = T' + S'_G + S'_{\text{PF}}$.
7. **Metropolis test**: accept with probability $\min(1, e^{-\Delta\mathcal{H}})$ where $\Delta\mathcal{H} = \mathcal{H}_f - \mathcal{H}_i$. If rejected, restore $U_\mu^{(0)}$.

9.3 Conjugate Momenta

The conjugate momenta $H_\mu(x)$ are traceless anti-Hermitian 3×3 matrices, elements of the Lie algebra $\mathfrak{su}(3)$. They have 8 real degrees of freedom. We generate them as follows:

- Off-diagonal: $H_{ij} = (a + ib)/\sqrt{2}$, $H_{ji} = (-a + ib)/\sqrt{2}$ with $a, b \sim \mathcal{N}(0, 1)$.
- Diagonal: $H_{ii} = id_i/\sqrt{2}$ with $d_i \sim \mathcal{N}(0, 1)$ for $i < N_c$, and $d_{N_c} = -\sum_{i < N_c} d_i$ (traceless).

The kinetic energy is

$$T = \frac{1}{2} \sum_{\mu,x} \sum_{i,j} |H_\mu(x)_{ij}|^2 = -\frac{1}{2} \sum_{\mu,x} \text{Tr}(H_\mu(x)^2). \quad (27)$$

Implementation: `conjugate_momentum.cpp`.

9.4 Pseudofermion Generation and Action

The pseudofermion field ϕ is generated by:

1. Draw a Gaussian random spinor field $\xi(x)$ with $\langle \xi_{s,c}^*(x) \xi_{s',c'}(y) \rangle = \delta_{xy} \delta_{ss'} \delta_{cc'}$.
2. Set $\phi = D_W^\dagger \xi$.

This ensures ϕ is distributed according to $P(\phi) \propto \exp(-\phi^\dagger (D_W^\dagger D_W)^{-1} \phi)$ since $\phi^\dagger (D_W^\dagger D_W)^{-1} \phi = \xi^\dagger D_W (D_W^\dagger D_W)^{-1} D_W^\dagger \xi = \xi^\dagger \xi$. Implementation: `generate_pseudofermion.cpp`.

The pseudofermion action at a general gauge field U (different from the one used to generate ϕ) is

$$S_{\text{PF}} = \phi^\dagger (D_W^\dagger D_W)^{-1} \phi = \text{Re}(\phi^\dagger X), \quad (28)$$

where X solves $D_W^\dagger D_W X = \phi$ via CG. Implementation: `pseudofermion_action.cpp`.

10 Molecular Dynamics Forces

10.1 Force Convention

The molecular dynamics forces enter the leapfrog integrator via the momentum update $H \leftarrow H - \epsilon F$. Our convention is that the force $F_\mu(x)$ is a traceless anti-Hermitian matrix satisfying

$$\left. \frac{dS}{d\epsilon} \right|_{U \rightarrow e^{\epsilon X} U} = -\text{Re Tr}(X \cdot F) \quad (29)$$

for any traceless anti-Hermitian direction X . Here $[\cdot]_{\text{TA}}$ denotes the traceless anti-Hermitian projection:

$$[M]_{\text{TA}} = \frac{M - M^\dagger}{2} - \frac{\text{Tr}(M - M^\dagger)}{2N_c} \cdot \mathbf{1}. \quad (30)$$

10.2 Gauge Force

The derivative of the Wilson gauge action (Eq. 5) with respect to link $U_\mu(x)$ in direction X gives:

$$\frac{dS_G}{d\epsilon} = -\frac{\beta}{N_c} \text{Re Tr}(X \cdot U_\mu V_\mu) = -\text{Re Tr}(X \cdot F_\mu^G), \quad (31)$$

where $V_\mu(x)$ is the staple sum (Eq. 7). Therefore:

$$F_\mu^G(x) = \frac{\beta}{N_c} [U_\mu(x) V_\mu(x)]_{\text{TA}} \quad (32)$$

Note: the staple V_μ enters *without* a dagger. Implementation: `gauge_force.cpp`.

10.3 Fermion Force

The derivative of the pseudofermion action (Eq. 28) with respect to $U_\mu(x)$ requires the solution $X = (D_W^\dagger D_W)^{-1} \phi$ (via CG) and $Y = D_W X$. The fermion force is:

$$F_\mu^F(x) = -2\kappa [U_\mu(x) M_\mu(x)]_{\text{TA}} \quad (33)$$

where the color matrix $M_\mu(x)$ is built from outer products of the fermion fields:

$$M_\mu(x)_{ab} = \sum_s \left[((r - \gamma_\mu)X(x + \hat{\mu}))_{s,a} Y^*(x)_{s,b} + Y(x + \hat{\mu})_{s,a} ((r + \gamma_\mu)X(x))_{s,b}^* \right]. \quad (34)$$

The first term arises from the forward hopping of D_W and the second from the backward hopping. The factor of 2κ (rather than κ) comes from matching the force convention (Eq. 29) with the inner product used by the gauge force. Implementation: `fermion_force.cpp`.

11 Leapfrog Integrator

11.1 Symplectic Integration

The leapfrog (Störmer–Verlet) scheme integrates Hamilton’s equations while preserving the symplectic structure. For N steps of size ϵ (total trajectory length $\tau = N\epsilon$):

$$\begin{aligned}
 H_{1/2} &= H_0 - \frac{\epsilon}{2} F(U_0) && \text{(half-step momentum)} \\
 \text{For } n &= 0, \dots, N-1 : \\
 U_{n+1} &= \exp(\epsilon H_{n+1/2}) \cdot U_n && \text{(full-step gauge)} \\
 H_{n+3/2} &= H_{n+1/2} - \epsilon F(U_{n+1}) && \text{(full-step momentum)} \\
 H_N &= H_{N-1/2} - \frac{\epsilon}{2} F(U_N) && \text{(final half-step)}
 \end{aligned} \tag{35}$$

where $F = F^G + F^F$ is the total force. The last full momentum step is omitted and replaced by the final half-step.

Key properties:

- **Symplectic:** preserves phase space volume exactly.
- **Time-reversible:** reversing all momenta and repeating returns to the initial state (to machine precision).
- **Energy conservation:** $\Delta\mathcal{H} = \mathcal{O}(\epsilon^2)$ per trajectory.

11.2 Gauge Field Update

The gauge field update $U \rightarrow \exp(\epsilon H) \cdot U$ requires the matrix exponential of a traceless anti-Hermitian matrix. We compute $\exp(A)$ via a 12-term Taylor series using Horner’s method:

$$\exp(A) \approx \mathbf{1} + A + \frac{A^2}{2!} + \dots + \frac{A^{12}}{12!}, \tag{36}$$

evaluated as $\exp(A) = \mathbf{1} + \frac{A}{12}(\mathbf{1} + \frac{A}{11}(\dots))$. The result is reunitarized to ensure exact SU(3) membership. For typical MD step sizes ($\epsilon \sim 0.02$), 12 terms gives machine-precision accuracy. Implementation: `exp_su3_algebra.cpp`.

11.3 Force Computation Cost

Each leapfrog step requires:

- One gauge force computation: $\mathcal{O}(V)$ staple calculations.

- One fermion force computation: one CG solve ($D_W^\dagger D_W X = \phi$) plus one D_W application, followed by $\mathcal{O}(V)$ outer product evaluations.

The fermion force (specifically the CG solve) dominates the computational cost, scaling as $\mathcal{O}(V \cdot N_{\text{CG}})$ per MD step. The total cost per trajectory is $\mathcal{O}(N_{\text{MD}} \cdot V \cdot N_{\text{CG}})$.

12 Code Architecture

12.1 Module Pattern

Global state is managed through module files following the Fortran module convention:

- `constants_module.hpp/.cpp`: Type definitions (`real_t`, `complex_t`), physical constants ($N_c = 3$, π).
- `su3_module.hpp/.cpp`: `SU3matrix` struct and all $\text{SU}(3)$ algebra operations. Thread-local RNG.
- `input_module.hpp/.cpp`: All simulation parameters as `extern` globals.
- `lattice_module.hpp/.cpp`: Lattice geometry, neighbor tables.
- `colorspinor_module.hpp/.cpp`: `ColorSpinor` struct and field operations.
- `gamma_matrices.hpp/.cpp`: Sparse Dirac gamma matrix representation.
- `interfaces_module.hpp`: Single header declaring all function prototypes.

12.2 Source File Organization

Each computational function resides in its own `.cpp` file. The complete file inventory:

File	Purpose
<i>Infrastructure</i>	
<code>lqcd_main.cpp</code>	Entry point: parse input, initialize, dispatch
<code>read_input_file.cpp</code>	INI-style parser (<code>[Section] / Key: Value</code>)
<code>set_parameter.cpp</code>	Map parsed key/value to global variables
<code>openmp_parallel_setup.cpp</code>	Thread count configuration
<i>Gauge Field Management</i>	
<code>initialize_gauge_field.cpp</code>	Cold (identity) or hot (random) start
<code>write_gauge_config.cpp</code>	Binary save with ASCII header
<code>read_gauge_config.cpp</code>	Binary load
<i>Pure Gauge Updates (Phase 1)</i>	
<code>compute_staple.cpp</code>	Staple sum $V_\mu(x)$
<code>heat_bath_sweep.cpp</code>	Cabibbo–Marinari heat bath
<code>overrelaxation_sweep.cpp</code>	Microcanonical reflection
<code>metropolis_sweep.cpp</code>	Metropolis update

File	Purpose
gauge_update_combined.cpp	1 HB + n_{OR} overrelaxation sweeps
thermalization.cpp	Thermalization loop
generate_configurations.cpp	Production loop with measurements
<i>Pure Gauge Measurements</i>	
compute_plaquette.cpp	Average plaquette
measure_plaquette.cpp	Plaquette measurement and output
wilson_loop.cpp	$W(R, T)$ for all R, T
polyakov_loop.cpp	Polyakov loop average
static_potential.cpp	$V(R)$ extraction
write_observables.cpp	ASCII output of measurements
jackknife_analysis.cpp	Delete-1 jackknife error estimation
<i>Wilson Fermions (Phase 2)</i>	
wilson_dirac_operator.cpp	$D_W \psi$
wilson_dirac_dagger.cpp	$D_W^\dagger \psi = \gamma_5 D_W \gamma_5 \psi$
even_odd_preconditioning.cpp	D_{eo}, D_{oe} , Schur complement
cg_solver.cpp	CG on $D_W^\dagger D_W$
bicgstab_solver.cpp	BiCGstab on D_W
point_source.cpp	Delta-function source
compute_propagator.cpp	12 inversions for full propagator
meson_correlator.cpp	$C_\Gamma(t)$ for 8 channels
effective_mass.cpp	Log ratio and cosh methods
write_correlators.cpp	ASCII correlator output
<i>HMC (Phase 3)</i>	
exp_su3_algebra.cpp	$\exp(\epsilon H) \cdot U$, TA projection, algebra norm
conjugate_momentum.cpp	Gaussian momentum generation, T
generate_pseudofermion.cpp	$\phi = D_W^\dagger \xi$
pseudofermion_action.cpp	$S_{\text{PF}} = \phi^\dagger (D_W^\dagger D_W)^{-1} \phi$
gauge_force.cpp	$F^G = (\beta/N_c)[UV]_{\text{TA}}$
fermion_force.cpp	$F^F = -2\kappa[UM]_{\text{TA}}$
leapfrog_integrator.cpp	Verlet integrator
hmc_trajectory.cpp	Full trajectory + Metropolis
hmc_driver.cpp	Thermalization + production

12.3 OpenMP Parallelization

The parallelization strategy varies by context:

- **Gauge updates:** Checkerboard (even/odd) decomposition. Even-parity links are updated in parallel, then odd-parity links. This avoids race conditions since no two even-parity links share a staple.
- **Measurements:** Trivially parallel over sites with `#pragma omp parallel for reduction(+:sum)`

- **Dirac operator:** Parallel over sites; each site computation is independent given the gauge and fermion fields.
- **Solver vector operations:** Parallel in dot products, norms, and axpy operations.
- **Thread-safe RNG:** Each thread has its own `std::mt19937_64` instance (`rng_local`), seeded from the master seed plus the thread ID. The seeding function `ensure_rng_seeded()` uses a thread-local flag to avoid redundant initialization.

12.4 Build System

The code is built via `src/Makefile` with `g++ -std=c++17 -O3 -march=native -fopenmp`. The wrapper script `build.sh` invokes `make -j$(nproc)`. Runtime scripts:

- `run.sh`: Set OMP environment, run executable, log timing.
- `batch_run.sh`: Sequential batch over multiple input files.
- `remote_run.sh`: Deploy, compile, run, and fetch results from a remote cluster.

13 Validation

13.1 Pure Gauge ($\beta = 6.0$, quenched)

- **4^4 lattice:** $\langle P \rangle \approx 0.59\text{--}0.60$ after thermalization (both hot and cold starts converge).
- **$8^3 \times 16$ lattice:** $\langle P \rangle \approx 0.593$. Wilson loops show area-law decay. Static potential $V(R)$ exhibits Coulomb behavior at short range and linear confinement at large R .
- **Polyakov loop:** $\langle L \rangle \approx 0$, consistent with the confined phase.

13.2 Wilson Fermions (quenched)

- **Free-field test** ($U_\mu = 1$): CG converges in 38 iterations, BiCGstab in 34. Pion correlator $C(0) = 17.72$, $m_{\text{eff}} = 0.419$.
- **Interacting** ($\beta = 6.0$, $\kappa = 0.15$): BiCGstab converges in ~ 100 iterations. Mass hierarchy $m_\pi < m_\rho < m_{\text{scalar}}$ confirmed.

13.3 HMC ($\beta = 5.6$, $\kappa = 0.12$)

- **Force validation:** Gauge and fermion forces match numerical derivatives of the action to 6+ digits.
- **Reversibility:** Forward + backward leapfrog returns to the initial state at machine precision ($\sim 10^{-15}$).
- **Energy conservation:** $\Delta\mathcal{H}$ scales as $\mathcal{O}(\epsilon^2)$ with MD step size ϵ .

- **Production:** 98% acceptance rate with $N_{\text{MD}} = 10$, $\epsilon = 0.02$. Plaquette $\langle P \rangle \approx 0.47$ – 0.48 . $\Delta\mathcal{H} \sim \mathcal{O}(0.01\text{--}0.1)$.

14 Wilson Flow and Scale Setting

14.1 Gradient Flow

The Wilson (gradient) flow [?] smoothly evolves the gauge field along a fictitious flow time t :

$$\frac{dV_\mu(x, t)}{dt} = -g_0^2 \frac{\partial S_W[V]}{\partial V_\mu(x, t)} \cdot V_\mu(x, t), \quad V_\mu(x, 0) = U_\mu(x), \quad (37)$$

where S_W is the Wilson plaquette action. The flow drives the field toward classical solutions, suppressing UV fluctuations at scales below $\sqrt{8t}$ while preserving long-distance physics. Composite operators evaluated on flowed fields are automatically renormalized [?].

14.2 Runge–Kutta Integration

We discretize the flow using Lüscher’s third-order Runge–Kutta (RK3) scheme [?]. Defining $Z = \epsilon \cdot (\beta/N_c)[V \cdot \text{staple}]_{\text{TA}}$ as the projected force, a single step $V \rightarrow V'$ proceeds as:

$$W_0 = V, \quad Z_0 = \epsilon S'(W_0) \quad (38)$$

$$W_1 = \exp\left(\frac{1}{4}Z_0\right) W_0, \quad Z_1 = \epsilon S'(W_1) \quad (39)$$

$$W_2 = \exp\left(\frac{8}{9}Z_1 - \frac{17}{36}Z_0\right) W_1, \quad Z_2 = \epsilon S'(W_2) \quad (40)$$

$$V' = \exp\left(\frac{3}{4}Z_2 - \frac{8}{9}Z_1 + \frac{17}{36}Z_0\right) W_2. \quad (41)$$

This scheme is $\mathcal{O}(\epsilon^3)$ accurate per step, time-reversible (negating ϵ returns to the original field up to machine precision), and preserves the gauge group exactly through the use of the matrix exponential. Implementation: `wilson_flow.cpp`.

14.3 Energy Density

We measure two definitions of the energy density at each flow time t :

$$E_{\text{plaq}}(t) = \frac{2}{V} \sum_{x, \mu > \nu} \left(1 - \frac{1}{N_c} \text{Re Tr } P_{\mu\nu}(x, t) \right), \quad (42)$$

$$E_{\text{clov}}(t) = -\frac{1}{V} \sum_{x, \mu > \nu} \text{Re Tr } F_{\mu\nu}(x, t)^2, \quad (43)$$

where $F_{\mu\nu}$ is the clover-definition field strength tensor (Section ??). The clover definition has $\mathcal{O}(a^2)$ -improved discretization errors and is preferred for precision scale setting.

14.4 Scale Setting: t_0 and w_0

The reference scale t_0 is defined by [?]:

$$t^2 \langle E(t) \rangle \Big|_{t=t_0} = 0.3. \quad (44)$$

The physical value $\sqrt{8t_0} \approx 0.3 \text{ fm}$ provides a precise non-perturbative scale. The related scale w_0 [?] is defined via the logarithmic derivative:

$$W(t) \equiv t \frac{d}{dt} [t^2 \langle E(t) \rangle] \Big|_{t=w_0^2} = 0.3. \quad (45)$$

The quantity w_0 has smaller statistical errors than $\sqrt{t_0}$ because the derivative suppresses long-range correlations. Both t_0/a^2 and w_0/a increase with β (finer lattice), enabling continuum extrapolation. Implementation: `analysis/scale_setting.py`.

15 Clover-Improved Wilson Fermions

15.1 $\mathcal{O}(a)$ Improvement

The standard Wilson action has discretization errors of $\mathcal{O}(a)$ arising from the dimension-5 Wilson term. The Sheikholeslami–Wohlert (clover) improvement [?] adds a dimension-5 counterterm that eliminates these errors:

$$D_{\text{clover}} = D_W + \frac{ic_{\text{SW}}}{4} \sum_{\mu < \nu} \sigma_{\mu\nu} \hat{F}_{\mu\nu}(x), \quad (46)$$

where $\sigma_{\mu\nu} = \frac{i}{2}[\gamma_\mu, \gamma_\nu]$ and c_{SW} is the improvement coefficient. With c_{SW} tuned appropriately (perturbatively or non-perturbatively), all on-shell observables are improved to $\mathcal{O}(a^2)$.

At tree level $c_{\text{SW}} = 1$. Non-perturbative determinations [?] give $c_{\text{SW}} \approx 1.769$ at $\beta = 6.0$ for quenched QCD.

15.2 Field Strength Tensor

The lattice field strength tensor $\hat{F}_{\mu\nu}(x)$ is constructed from the four-leaf clover:

$$Q_{\mu\nu}(x) = P_{\mu,\nu}(x) + P_{\nu,-\mu}(x) + P_{-\mu,-\nu}(x) + P_{-\nu,\mu}(x), \quad (47)$$

where $P_{\mu,\nu}(x)$ denotes the plaquette starting at x in the (μ, ν) direction, and the other three terms are the plaquettes in the remaining orientations sharing the point x . The field strength is obtained by anti-Hermitian projection:

$$\hat{F}_{\mu\nu}(x) = \frac{Q_{\mu\nu}(x) - Q_{\mu\nu}^\dagger(x)}{8}. \quad (48)$$

This is automatically traceless and anti-Hermitian. The same $\hat{F}_{\mu\nu}$ is used both in the clover Dirac operator and in the clover definition of the energy density (Eq. ??). Implementation: `compute_field_strength_tensor()` in `wilson_flow.cpp`, reused by `clover_term.cpp`.

15.3 Block Structure

In the DeGrand–Rossi basis, the $\sigma_{\mu\nu}$ matrices are block-diagonal: they act within the upper block (spin 0,1) and lower block (spin 2,3) independently. Combined with $N_c = 3$ colors, the clover term at each site consists of two Hermitian 6×6 matrices (upper and lower blocks in spin-color space):

$$A(x) = \frac{ic_{\text{SW}}}{4} \sum_{\mu < \nu} \sigma_{\mu\nu} \otimes \hat{F}_{\mu\nu}(x) = \begin{pmatrix} A^{(+)}(x) & 0 \\ 0 & A^{(-)}(x) \end{pmatrix}. \quad (49)$$

This block structure is exploited for storage efficiency and enables the clover term to be applied with $2 \times 6^2 = 72$ multiply-adds per site rather than the $12^2 = 144$ required for a general 12×12 matrix.

Data structures: `CloverBlock` (one 6×6 complex matrix), `CloverField` (upper + lower blocks per site), defined in `clover_module.hpp`.

15.4 Even–Odd Preconditioning with Clover

With the clover term, the diagonal blocks of the Dirac operator become non-trivial:

$$D_{\text{clover}} = \begin{pmatrix} 1 + A_{ee} & -\kappa D_{eo} \\ -\kappa D_{oe} & 1 + A_{oo} \end{pmatrix}, \quad (50)$$

and the preconditioned operator becomes:

$$\hat{D} = 1 - \kappa^2 (1 + A_{ee})^{-1} D_{eo} (1 + A_{oo})^{-1} D_{oe}. \quad (51)$$

The inversions $(1 + A_{ee/oo})^{-1}$ are performed as 6×6 dense matrix inversions (pivoted LU decomposition) at each site, precomputed before the solver. Implementation: `clover_even_odd.cpp`, `invert_clover_block()` in `clover_term.cpp`.

15.5 Dispatch Wrappers

To avoid modifying the solver interfaces, the clover Dirac operator is accessed through dispatch wrappers:

- `apply_dirac()`: checks the global flag `useClover` and routes to either `wilson_dirac_operator()` or `clover_dirac_operator()`.
- `apply_dirac_dagger()`: analogous for the adjoint.

The wrappers manage an internal cache of the `CloverField` that is recomputed when the gauge field changes (signaled by `invalidate_clover_cache()` after each gauge field update in the leapfrog integrator). Implementation: `clover_dirac_operator.cpp`.

15.6 Clover Fermion Force

For HMC with clover improvement, the molecular dynamics force acquires an additional contribution from the derivative of the clover term with respect to the gauge links:

$$F_{\mu}^{\text{clover}}(x) = -c_{\text{SW}} \sum_{\nu \neq \mu} \frac{\partial \hat{F}_{\mu\nu}}{\partial U_{\mu}(x)} \cdot \frac{\partial S_{\text{PF}}}{\partial \hat{F}_{\mu\nu}}. \quad (52)$$

The derivative $\partial \hat{F}_{\mu\nu} / \partial U_{\mu}(x)$ involves staple-like contributions from the four leaves of the clover that pass through the link $U_{\mu}(x)$. The site-local factor $\partial S_{\text{PF}} / \partial \hat{F}_{\mu\nu}$ is constructed from the solution vectors $X = (D^{\dagger} D)^{-1} \phi$ and $Y = DX$. This force is added to the Wilson hopping force in the leapfrog integrator. Implementation: `clover_fermion_force.cpp`.

16 Autocorrelation Analysis

16.1 Integrated Autocorrelation Time

Monte Carlo measurements are correlated in Markov chain time. The normalized autocorrelation function of an observable $\mathcal{O}$ is:

$$C(t) = \frac{\langle \mathcal{O}_i \mathcal{O}_{i+t} \rangle - \langle \mathcal{O} \rangle^2}{\langle \mathcal{O}^2 \rangle - \langle \mathcal{O} \rangle^2}, \quad (53)$$

and the integrated autocorrelation time is:

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{t=1}^{\infty} C(t). \quad (54)$$

The effective number of independent measurements is $N_{\text{eff}} = N / (2\tau_{\text{int}})$, and the true statistical error is a factor $\sqrt{2\tau_{\text{int}}}$ larger than the naive error.

16.2 Madras–Sokal Windowing

The sum in Eq. (??) must be truncated at a finite window W . The naive approach (truncate at the first negative $C(t)$) systematically underestimates τ_{int} when the autocorrelation function oscillates or has a long tail.

The Madras–Sokal self-consistent windowing procedure [?] determines W by requiring:

$$W = c \cdot \tau_{\text{int}}(W), \quad c \approx 6, \quad (55)$$

where $\tau_{\text{int}}(W) = \frac{1}{2} + \sum_{t=1}^W C(t)$ is the running estimate. The smallest W satisfying this self-consistency condition is used. The constant $c \approx 6$ balances the bias from truncation against the variance from including noisy estimates of $C(t)$ at large t .

16.3 Binned Jackknife

Given τ_{int} , the optimal jackknife bin size is $B = \lceil 2\tau_{\text{int}} \rceil$. The binned jackknife procedure:

1. Divide the N measurements into $N_B = \lfloor N/B \rfloor$ bins, each containing B consecutive measurements averaged together.
2. Apply the standard delete-1 jackknife to the N_B binned values.

For sufficiently large B , the binned data are approximately independent and the jackknife error is unbiased. The error ratio $\sigma_{\text{binned}}/\sigma_{\text{naive}} \geq 1$ provides a diagnostic: values significantly greater than 1 indicate that naive errors are underestimated.

Implementation: `analysis/autocorrelation_analysis.py` provides the Madras–Sokal windowing and generates diagnostic plots. The analysis scripts `spectroscopy.py` and `static_potential.py` use binned jackknife with the measured optimal bin size.

References

- [1] K. G. Wilson, “Confinement of quarks,” *Phys. Rev. D* **10**, 2445 (1974).
- [2] K. G. Wilson, “Quarks and strings on a lattice,” in *New Phenomena in Subnuclear Physics* (Plenum, New York, 1977).
- [3] N. Cabibbo and E. Marinari, “A new method for updating $SU(N)$ matrices in computer simulations of gauge theories,” *Phys. Lett. B* **119**, 387 (1982).
- [4] A. D. Kennedy and B. J. Pendleton, “Improved heat bath method for Monte Carlo calculations in lattice gauge theory,” *Phys. Lett. B* **156**, 393 (1985).
- [5] S. Duane, A. D. Kennedy, B. J. Pendleton, and D. Roweth, “Hybrid Monte Carlo,” *Phys. Lett. B* **195**, 216 (1987).
- [6] H. A. van der Vorst, “Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems,” *SIAM J. Sci. Stat. Comput.* **13**, 631 (1992).
- [7] T. DeGrand and C. DeTar, *Lattice Methods for Quantum Chromodynamics* (World Scientific, Singapore, 2006).
- [8] C. Gattringer and C. B. Lang, *Quantum Chromodynamics on the Lattice* (Springer, Berlin, 2010).
- [9] H. J. Rothe, *Lattice Gauge Theories: An Introduction*, 4th ed. (World Scientific, Singapore, 2012).
- [10] M. Lüscher, “Properties and uses of the Wilson flow in lattice QCD,” *JHEP* **1008**, 071 (2010) [arXiv:1006.4518].
- [11] M. Lüscher and P. Weisz, “Perturbative analysis of the gradient flow in non-abelian gauge theories,” *JHEP* **1102**, 051 (2011) [arXiv:1101.0963].

- [12] S. Borsanyi *et al.* (BMW Collaboration), “High-precision scale setting in lattice QCD,” JHEP **1209**, 010 (2012) [arXiv:1203.4469].
- [13] B. Sheikholeslami and R. Wohlert, “Improved continuum limit lattice action for QCD with Wilson fermions,” Nucl. Phys. B **259**, 572 (1985).
- [14] M. Lüscher, S. Sint, R. Sommer, and H. Wittig, “Non-perturbative determination of the axial current normalization constant in $O(a)$ improved lattice QCD,” Nucl. Phys. B **491**, 344 (1997) [arXiv:hep-lat/9611015].
- [15] N. Madras and A. D. Sokal, “The pivot algorithm: A highly efficient Monte Carlo method for the self-avoiding walk,” J. Stat. Phys. **50**, 109 (1988).